THE PREDICTION OF CEREBRAL PALSY IN INFANTS USING POSE ESTIMATION TECHNIQUES WITHIN A HYBRID DEEP LEARNING FRAMEWORK

A project report submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING (AI & ML)

by

T. ROHITH SAI	22BFA33321
K. GNANANJALI	23BFA33L11
B. KIRAN KUMAR	22BFA33285
B. GURU VIGNESH	22BFA33286

Under the guidance of

Dr. P. LOKESH KUMAR REDDY M.Tech., P.hD

Associate Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI & ML)

SRI VENKATESWARA COLLEGE OF ENGINEERING

(AUTONOMOUS)

(Approved by AICTE, New Delhi & Permanently Affiliated to JNTUA, Ananthapuramu
Accredited by NBA, New Delhi & NAAC with 'A' grade)
Karakambadi Road, TIRUPATI – 517507

2022 - 2026

SRI VENKATESWARA COLLEGE OF ENGINEERING

(AUTONOMOUS)

(Approved by AICTE, New Delhi & PermanentlyAffiliated to JNTUA, Ananthapuramu
Accredited by NBA, New Delhi & NAACwith 'A' Grade)
Karakambadi Road, TIRUPATI – 517507.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI & ML)

CERTIFICATE

This is to certify that the project work entitled, **"THE PREDICTION OF CEREBRAL PALSY IN INFANTS USING POSE ESTIMATION TECHNIQUES WITHIN A HYBRID DEEP LEARNING FRAMEWORK"** is a bonafide record of the project work done and submitted by

T. ROHITH SAI	**22BFA33321**
K. GNANANJALI	**23BFA33L11**
B. KIRAN KUMAR	**22BFA33285**
B. GURU VIGNESH	**22BFA33286**

under my guidance and supervision for the partial fulfillment of the requirements for the award of B.Tech degree in **COMPUTER SCIENCE AND ENGINEERING (AI & ML).** This project is the result of our own effort and that it has not been submitted to any other University or Institution for the award of any degree or diploma other than specified above

GUIDE **HEAD OF THE DEPARTMENT**

External Viva-Voce Exam held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DECLARATION

We hereby declare that the project entitled **"THE PREDICTION OF CEREBRAL PALSY IN INFANTS USING POSE ESTIMATION TECHNIQUES WITHIN A HYBRID DEEP LEARNING FRAMEWORK",** done by us under the esteemed guidance of **Dr. P. Lokesh Kumar Reddy,** Professor and is submitted in partial fulfillment of the requirements for the award of the Degree of **Bachelor of Technology in Computer Science and Engineering (AI & ML).**

This report is the result of our own effort and it has not been submitted to any other University or Institution for the award of any degree or diploma other than specified above.

Date:

Place:

T. ROHITH SAI	**22BFA33321**
K. GNANANJALI	**23BFA33L11**
B. KIRAN KUMAR	**22BFA33285**
B. GURU VIGNESH	**22BFA33286**

ACKNOWLEDGEMENT

We are thankful to our guide **Dr. P. Lokesh Kumar Reddy, Associate Professor, Department of CSE (AI&ML)** for his valuable guidance and encouragement. His helping attitude and suggestions have helped in the successful completion of the project report.

We would like to express our gratefulness and sincere thanks to **Dr. R. Swathi, Professor & HOD, Department of CSE (AI&ML),** for her kind help and encouragement during the course of study and in the successful completion of the project work.

We have great pleasure in expressing out hearty thanks to our beloved Principal **Dr. Vijaya Gunturu** for spending his valuable time with us to complete this project.

Successful completion of any project cannot be done without proper support and encouragement. We sincerely thank the management for providing all the necessary facilities during course of study.

We would like to thank our parents and friends, who have the greatest contributions in all our achievements, for the great care and blessings in making us successful in all our endeavours.

T. ROHITH SAI **22BFA33321**

K. GNANANJALI **23BFA33L11**

B. KIRAN KUMAR **22BFA33285**

B. GURU VIGNESH **22BFA33286**

ABSTRACT

Cerebral Palsy (CP) is a neurological disorder that affects movement, posture, and muscle coordination in infants. Early detection is crucial for effective intervention; however, traditional diagnostic methods rely heavily on subjective clinical observation, leading to delayed identification.

This project proposes an automated and objective system for early prediction of cerebral palsy using video-based movement analysis. Infant movement videos are processed using pose estimation techniques to extract skeletal keypoints representing body joints. These keypoints are transformed into structured temporal data and analyzed using hybrid deep learning models, including CNN-LSTM, CNN-GRU, and GCN-GRU architectures.

The system captures both spatial and temporal movement patterns to identify subtle abnormalities that are difficult to detect manually. An ensemble learning approach based on majority voting is applied to improve prediction accuracy and robustness.

Experimental results demonstrate that the proposed system provides reliable and scalable predictions, supporting early diagnosis and assisting clinicians in decision-making. This approach has the potential to enhance early intervention strategies and improve long-term outcomes for infants at risk of cerebral palsy.

CONTENTS

Abstract	**i**
Table of Contents	**ii**
List of figures	**iv**
List of Tables	**v**
Abbreviations	**vi**

Chapter No.	Description	Page No.
1	**INTRODUCTION**	**1**
2	**PROJECT DESCRIPTION**	**2-4**
	2.1 Problem Definition	2
	2.2 Project Details	2
	2.3 Scope of the Project	3
3	**SYSTEM ANALYSIS**	**5-7**
	3.1 Existing System	5
	3.1.1 Disadvantages	5
	3.2 Proposed System	6
	3.2.1 Advantages	7

Chapter No.	Description	Page No.
4	**ENVIRONMENTS**	**8-12**
	4.1 Software Requirements.	8
	4.2 Hardware Requirements	8
	4.3 Software Features and Dependencies	8
5	**SYSTEM DESIGN**	**13-28**
	5.1 System Architecture	13
	5.2 UML Diagrams	16
	5.2.1 Class Diagram	17
	5.2.2 Use Case Diagram	19
	5.2.3 Sequence Diagram	22
	5.2.4 Activity Diagram	24
	5.2.5 Component Diagram	26
	5.2.6 Deployment Diagram	27
6	**SYSTEM IMPLEMENTATION**	**29-33**
	6.1 Implementation Process	29
	6.2 Module Explanation	31
7	**SYSTEM STUDY AND TESTING**	**34-37**
	7.1 Feasibility Study	34
	7.2 Software Testing	35

Chapter No.	Description	Page No.
8	SOURCE CODE	38
9	SCREEN LAYOUTS	47
10	FUTURE ENHANCEMENTS	51
11	CONCLUSION	53
12	BIBILOGRAPHY	54

LIST OF FIGURES

Fig. No.	Figure Name	Page No.
5.1	System Architecture	15
5.2.1	Class diagram	19
5.2.2	Use Case diagram	22
5.2.3	Sequence diagram	24
5.2.4	Activity diagram	26
5.2.5	Component diagram	27
5.2.6	Deployment diagram	28
9.1	Interface Layout	47
9.1.1	User Registration Page	47
9.1.2	User Login Page	48
9.1.3	Video upload and prediction interface	48
9.1.4	Choosing the Video file for upload	49
9.1.5	Loading Page	49
9.2.1	Output 1	50
9.2.2	Output 2	50

LIST OF TABLES

Table No.	Table Name	Page No.
7.2.1	Unit testing	35
7.2.2	Integration testing	35
7.2.3	System testing	36
7.2.4	Acceptance testing	37

ABBREVIATIONS

AI	Artificial Intelligence
Bi-LSTM	Bidirectional Long Short-Term Memory
CNN	Convolutional Neural Network
CP	Cerebral Palsy
CV	Computer Vision
DL	Deep Learning
Flask	Lightweight Python Web Framework
FPS	Frames Per Second
GPU	Graphics Processing Unit
GUI	Graphical User Interface
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
LSTM	Long Short-Term Memory
ML	Machine Learning
MP	MediaPipe
MSE	Mean Squared Error
NLP	Natural Language Processing
NumPy	Numerical Python Library
OpenCV	Open Source Computer Vision Library
ORM	Object Relational Mapping
Pandas	Python Data Analysis Library
PIL	Python Imaging Library
RNN	Recurrent Neural Network
SQL	Structured Query Language

SQLite	Lightweight Database System
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
CPU	Central Processing Unit
CSV	Comma Separated Values
XML	eXtensible Markup Language
API	Application Programming Interface
IoT	Internet of Things

CHAPTER 1

INTRODUCTION

Cerebral Palsy (CP) is a group of permanent neurological disorders that affect movement, posture, and muscle coordination, primarily caused by abnormal brain development or early brain injury. It is one of the most common motor disabilities in children, with significant impact on physical and cognitive development. Early detection of CP is critical, as timely therapeutic intervention can greatly improve motor function, reduce complications, and enhance the overall quality of life of affected infants.

However, early diagnosis of cerebral palsy remains a major clinical challenge. In the initial months of life, symptoms are often subtle, inconsistent, and difficult to interpret. Traditional diagnostic approaches rely on clinical observation, neurological examinations, and General Movement Assessment (GMA), which depend heavily on expert interpretation. These methods are subjective, time-consuming, and not easily scalable, especially in regions with limited access to specialized healthcare professionals.

With the advancement of Artificial Intelligence (AI) and Computer Vision, there is a growing opportunity to develop automated systems capable of analyzing infant movement patterns objectively. These keypoints provide a structured and quantifiable representation of movement, which can be effectively analyzed using deep learning models.

This project proposes an intelligent framework for the early prediction of cerebral palsy using video-based movement analysis. Infant videos are processed using pose estimation to extract key joint coordinates, which are then preprocessed and transformed into sequential data. Hybrid deep learning models, such as CNN-LSTM, CNN-GRU, and GCN-GRU, are employed to capture both spatial relationships between joints and temporal movement patterns across frames.

The proposed system aims to provide an automated, objective, and scalable solution for early CP detection. By assisting clinicians with data-driven insights, it has the potential to reduce diagnostic delays, improve early intervention strategies, and contribute to better long-term developmental outcomes for infants.

CHAPTER 2

PROJECT DESCRIPTION

2.1 PROBLEM DEFINITION:

Cerebral Palsy (CP) is often diagnosed at a later stage due to the subtle and highly variable nature of early symptoms in infants. In the initial months, abnormal movement patterns are difficult to identify and require expert interpretation. Traditional assessment methods, such as manual observation of infant videos and General Movement Assessment (GMA), are subjective, time-consuming, and dependent on specialized clinicians, making early screening inaccessible in many healthcare settings.

To address these limitations, there is a need for an automated, objective, and scalable system capable of analyzing infant movement patterns efficiently. By leveraging pose estimation techniques to extract skeletal keypoints from video data and applying hybrid deep learning models to analyze spatiotemporal patterns, the proposed system aims to accurately predict early signs of cerebral palsy and assist clinicians in timely diagnosis and intervention.

2.2 PROJECT DETAILS

This project focuses on developing an intelligent system for the early prediction of Cerebral Palsy (CP) in infants using pose estimation and hybrid deep learning techniques. The system analyzes infant movement patterns from video data to provide an automated and objective alternative to traditional clinical assessments.

The process begins with collecting infant movement videos from the RVI-38 dataset, which includes recordings of infants aged between three to five months. These videos are processed using the OpenPose framework to extract skeletal keypoints corresponding to major body joints such as the head, shoulders, elbows, hips, knees, and ankles. Out of the 25 detected keypoints, 15 significant joints are selected to represent meaningful motor activity. This step converts raw video data into structured numerical sequences suitable for machine learning.

The extracted keypoints undergo preprocessing to enhance data quality. Missing or low-confidence values are corrected using interpolation techniques, noise is reduced using smoothing filters, and normalization is applied to eliminate variations caused by camera positioning. Data augmentation is performed by shifting keypoint coordinates to generate additional training samples, improving model robustness and generalization.

The processed data is then used to train multiple deep learning models. Both basic models (RNN, GRU, LSTM, CNN) and hybrid models (CNN-LSTM, CNN-GRU, and GCN-GRU) are implemented. These hybrid architectures effectively capture spatial relationships between body joints and temporal movement patterns across frames. Regularization techniques such as dropout, L1/L2 regularization, and batch normalization are applied to prevent overfitting.

To further improve prediction performance, an ensemble learning approach based on majority voting is employed. Predictions from multiple hybrid models are combined to produce a final output, increasing accuracy and reducing model bias. The ensemble model demonstrates superior performance, highlighting the effectiveness of combining multiple deep learning approaches for CP prediction.

2.3 SCOPE OF THE PROJECT

The scope of this project is centered on developing an automated system for the early prediction of Cerebral Palsy (CP) in infants using video-based movement analysis. The system specifically targets infants aged between three to five months, focusing on detecting subtle abnormalities in motor behavior through pose estimation and deep learning techniques.

This project has significant applications in the healthcare domain, particularly in pediatric neurology. It assists clinicians by providing objective and data-driven insights into infant movement patterns, reducing reliance on subjective assessments. The integration of pose estimation and hybrid deep learning enables the detection of fine-grained spatiotemporal abnormalities that may not be easily observable through manual evaluation.

The system can be extended for real-time applications, where infant movements can be analyzed using mobile devices or web-based platforms. This enables remote monitoring and increases accessibility, especially in rural or resource-limited regions. Caregivers could record videos at home and receive preliminary risk assessments, facilitating early medical consultation.

From a research perspective, the project provides opportunities for further enhancement, including the use of larger and more diverse datasets, adoption of 3D pose estimation techniques, and development of lightweight models for real-time deployment. Additionally, integrating explainable AI techniques can improve interpretability and trust among healthcare professionals.

However, the current scope is limited by dataset size, computational requirements, and the need for clinical validation. Despite these limitations, the system establishes a strong foundation for intelligent healthcare solutions aimed at early disease detection and improved patient outcomes.

CHAPTER 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

In the existing system, the diagnosis of cerebral palsy (CP) primarily relies on clinical observation, neurological examinations, and developmental milestone tracking over time. Pediatric neurologists evaluate infants based on motor delays, abnormal muscle tone, reflex behavior, and posture to identify potential signs of CP.

In addition to clinical assessments, techniques such as General Movement Assessment (GMA) are used to analyze infant movements through visual observation. Imaging methods like MRI and CT scans are also employed to detect structural abnormalities in the brain; however, these are typically performed only after symptoms become noticeable.

Despite their clinical importance, these approaches have several limitations. Early symptoms of CP are often subtle and difficult to detect in the first few months of life, which leads to delayed diagnosis and intervention. The process is highly dependent on expert interpretation, making it subjective and prone to variability across clinicians. Furthermore, manual assessment of infant movement is time-consuming and not scalable, especially in regions with limited access to specialized healthcare professionals.

3.1.1 Disadvantages

- ***Highly subjective:***
  Diagnosis depends on expert interpretation and may vary between clinicians.
- ***Time-consuming:***
  Manual observation and video analysis delay early detection.
- ***Limited accessibility:***
  Requires trained specialists, making early screening unavailable in many areas.
- ***Low scalability:***
  Traditional methods cannot efficiently handle large populations.

- ***Inconsistent detection:***

  Subtle movement abnormalities may be overlooked.
- ***Lack of automation:***

  Existing approaches do not effectively utilize AI for continuous and objective analysis.

3.2 PROPOSED SYSTEM

The proposed system introduces an automated and intelligent framework for the early prediction of cerebral palsy in infants using video-based movement analysis. The system aims to overcome the limitations of traditional methods by providing an objective, scalable, and data-driven solution.

The system takes infant movement videos as input and processes them using a pose estimation technique (OpenPose) to extract skeletal keypoints representing major body joints such as the head, shoulders, elbows, hips, knees, and ankles. These keypoints represent the spatial structure of the infant's posture across video frames and are converted into structured sequential data.

The extracted keypoints undergo preprocessing to enhance data quality and reliability. Missing or low-confidence keypoints are corrected using interpolation techniques, noise is reduced using smoothing filters, and normalization is applied to eliminate variations caused by camera positioning. Data augmentation techniques are also applied to improve model generalization and handle limited data.

The processed sequential data is then fed into a Bidirectional Long Short-Term Memory (Bi-LSTM) model for classification. The Bi-LSTM model is capable of learning temporal dependencies in both forward and backward directions, allowing it to capture complete movement patterns over time. This makes it highly effective in identifying subtle abnormalities in infant motor behavior that are difficult to detect using traditional approaches.

Based on the learned patterns, the system predicts whether an infant is at risk of cerebral palsy or shows normal development. The prediction results are presented through a user interface, assisting clinicians in early diagnosis and enabling timely intervention.

3.2.1 Advantages

- ***Early prediction***

  Enables detection of CP at an early developmental stage.
- ***Bidirectional temporal learning:***

  Bi-LSTM captures both past and future movement dependencies for improved accuracy.
- ***Objective analysis:***

  Reduces reliance on subjective clinical observation.
- ***Detection of subtle abnormalities:***

  Identifies complex movement patterns not easily visible to the human eye.
- ***Scalable solution:***

  Can be deployed in hospitals, clinics, and remote healthcare environments.
- ***Faster diagnosis:***

  Reduces the time required for manual analysis.
- ***Decision support:***

  Assists clinicians in making informed and timely medical decisions.

CHAPTER 4

ENVIRONMENTS

4.1 SOFTWARE REQUIREMENTS

- Operating System : Windows XP/7/8/10
- Programming language : Python 3.8 – 3.11.
- Web-Server : Flask Web Server
- Front end : HTML, CSS, Java Script
- Back end : MySQL Database
- Deep learning : PyTorch (preferred)

4.2 HARDWARE REQUIREMENTS

- Processor : Intel Core i5 (6-core) or AMD Ryzen 5
- RAM : 16 GB recommended (8 GB minimum).
- Hard disk : 250 GB SSD
- Monitor : Standard display
- GPU : NVIDIA GPU with CUDA support

4.3 SOFTWARE FEATURES AND DEPENDENCIES

The proposed system relies on a combination of software libraries, frameworks, and tools to perform video processing, pose estimation, data preprocessing, deep learning model training, and deployment. Each dependency plays a specific role in the overall pipeline of the system.

Python

Python is the primary programming language used for the development of the system. It is widely adopted in machine learning and computer vision applications due to its simplicity, readability, and extensive ecosystem of libraries. Python enables rapid

prototyping, efficient data handling, and seamless integration with deep learning frameworks.

Key Features:

- High-level and easy-to-learn syntax
- Extensive support for scientific computing and AI
- Large community and open-source ecosystem
- Cross-platform compatibility

OpenPose

OpenPose is a real-time multi-person pose estimation framework used to extract skeletal keypoints from video data. It detects human body joints such as the head, shoulders, elbows, hips, knees, and ankles and provides their spatial coordinates.

Role in System:

- Converts raw infant videos into structured keypoint data
- Extracts spatial features required for movement analysis
- Provides confidence scores for each detected keypoint

Key Features:

- Real-time pose detection
- Multi-body keypoint extraction
- High accuracy in joint localization
- Supports 2D skeletal tracking

OpenCV

OpenCV (Open Source Computer Vision Library) is used for video and image processing. It plays a critical role in handling input video data before pose estimation.

Role in System:

- Reading and processing video files
- Extracting frames from video sequences
- Performing basic image transformations and preprocessing

Key Features:

- Efficient real-time video processing
- Supports multiple image and video formats

NumPy

NumPy is a fundamental library for numerical computing in Python. It is used for handling arrays and performing mathematical operations on keypoint data.

Role in System:

- Storing and manipulating keypoint coordinates
- Performing vectorized computations
- Supporting preprocessing operations

Key Features:

- Fast array operations
- Memory-efficient data structures
- Integration with other scientific libraries

Pandas

Pandas is used for data manipulation and preprocessing. It provides structured data handling through DataFrames.

Role in System:

- Organizing keypoint data into structured format
- Handling missing values and data cleaning
- Preparing datasets for model training

Key Features:

- Easy data manipulation
- Handling large datasets efficiently
- Support for CSV and database integration

SciPy

SciPy provides advanced mathematical and scientific computing functions.

Role in System:

- Interpolation of missing keypoints
- Signal smoothing and filtering
- Mathematical operations during preprocessing

Key Features:

- Advanced numerical algorithms
- Integration with NumPy
- Efficient scientific computations

PyTorch

PyTorch is the core deep learning framework used for building and training the Bidirectional Long Short-Term Memory (Bi-LSTM) model.

Role in System:

- Designing and training the Bi-LSTM architecture
- Handling sequential data modeling
- Performing forward and backward propagation

Key Features:

- Dynamic computational graph (flexibility in model design)
- GPU acceleration for faster training
- Automatic differentiation (autograd)
- Strong support for sequence-based models

TorchVision

TorchVision is a PyTorch library that provides tools for handling image and video datasets.

Role in System:

- Dataset handling and transformations
- Preprocessing support for image-based inputs

Key Features:

- Pre-built utilities for vision tasks
- Integration with PyTorch pipelines

Matplotlib and Seaborn

These libraries are used for data visualization and performance analysis.

Role in System:

- Visualizing training metrics (loss, accuracy)
- Plotting graphs for model evaluation
- Analyzing prediction results

Key Features:

- Easy plotting and customization
- Statistical visualization support

Flask

Flask is a lightweight web framework used to deploy the trained model as a web application.

Role in System:

- Creating user interface for video upload
- Connecting frontend with backend model
- Displaying prediction results to users

Key Features:

- Simple and lightweight framework
- Easy routing and request handling
- Flexible integration with ML models

JSON and Pickle

These are used for data storage and serialization.

Role in System:

- Saving trained models and configurations
- Storing prediction results
- Enabling efficient data exchange.

CHAPTER 5

SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE

System design refers to the structured organization and integration of various components to achieve the desired functionality of a software system efficiently. For the project titled "Prediction of Cerebral Palsy in Infants using Pose Estimation and Deep Learning," the system architecture is designed as a modular pipeline that processes infant movement videos and predicts the risk of cerebral palsy using advanced machine learning techniques. The proposed system follows a sequential pipeline consisting of multiple interconnected modules, including video input, frame extraction, pose estimation, preprocessing, feature modeling using Bidirectional Long Short-Term Memory (Bi-LSTM), and prediction output.

1. Video Input Module

The system begins with the input of infant movement videos. These videos may be sourced from datasets such as RVI-38 or uploaded by users through a web interface. The videos contain natural movement patterns of infants, which are essential for detecting abnormalities in motor behavior.

2. Frame Extraction Module

The input video is processed using video handling techniques to extract individual frames. Each video is divided into a sequence of frames to enable temporal analysis of movement patterns over time. This step ensures that motion information is preserved in a structured format.

3. Pose Estimation Module

Each frame is passed through a pose estimation framework (OpenPose), which extracts skeletal keypoints representing major body joints such as the head, shoulders, elbows, hips, knees, and ankles.

The output of this module is a set of keypoints for each frame, where each keypoint contains spatial coordinates (x, y) and a confidence score. These keypoints transform raw visual data into structured numerical data suitable for machine learning.

4. Data Preprocessing Module

The extracted keypoint data is often noisy and incomplete. Therefore, preprocessing is performed to improve data quality:

- Missing keypoints are filled using interpolation techniques
- Noise is reduced using smoothing filters
- Normalization is applied to remove variations due to camera angle and positioning
- Data augmentation is performed by slightly shifting keypoints to improve model generalization

This module ensures that the data is clean, consistent, and ready for model training.

5. Feature Representation Module

The processed keypoints are converted into sequential data representing movement over time. Each sequence contains temporal information about joint positions across frames, which is essential for understanding motion patterns.

6. Bi-LSTM Model Module

The sequential data is fed into a Bidirectional Long Short-Term Memory (Bi-LSTM) model. This model is designed to capture temporal dependencies in both forward and backward directions.

Unlike traditional models, Bi-LSTM analyzes the entire sequence context, allowing it to detect subtle abnormalities in infant movement patterns. This makes it highly effective for time-series analysis of motion data.

7. Prediction Module

The trained Bi-LSTM model outputs a classification result indicating whether the infant is:

- At risk of Cerebral Palsy
- Normal (no risk detected)

The prediction is based on learned patterns from training data and is generated in real time.

8. User Interface Module

The final output is displayed through a web-based user interface developed using Flask. Users can upload videos and view prediction results easily. The interface is designed to be simple and user-friendly for clinicians and caregivers.

Overall System Flow

Video Input → Frame Extraction → Pose Estimation → Keypoint Extraction → Preprocessing → Sequence Formation → Bi-LSTM Model → Prediction → User Interface

System Characteristics

- Modular design: Each component operates independently and can be updated without affecting the entire system
- Scalability: Can be extended to real-time systems and mobile applications
- Accuracy: Uses deep learning for precise prediction
- Automation: Reduces manual effort in diagnosis
- Flexibility: Can integrate with healthcare platforms

In conclusion, the proposed system architecture provides an efficient and scalable pipeline for analyzing infant movement patterns and predicting cerebral palsy at an early stage. By integrating pose estimation and Bi-LSTM modeling, the system ensures accurate, objective, and real-time predictions, supporting early diagnosis and intervention.

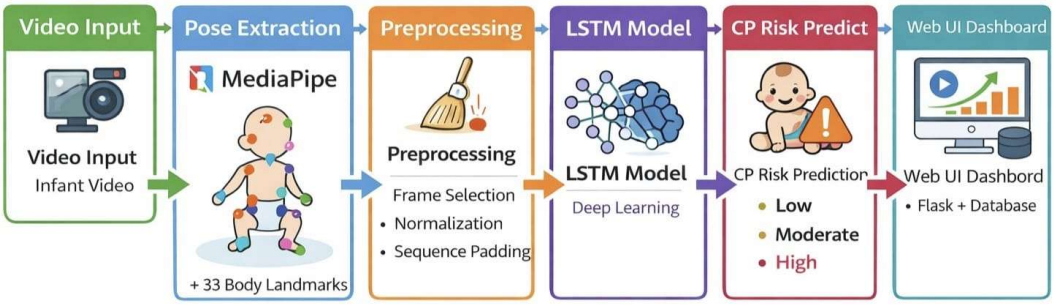

Fig 5.1: System architecture

5.2 UML DIAGRAMS

What is UML?

The Unified Modelling Language (UML) is a standard language for specifying, visualizing, constructing, and documenting the artifacts of software systems, as well as for business modelling and other non-software systems. The UML represents a collection of best engineering practices that have proven successful in the modelling of large and complex systems. The UML is a very important part of developing objects-oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects. Using the UML helps project teams communicate, explore potential designs, and validate the architectural design of the software.

As the strategic value of software increases for many companies, the industry looks for techniques to automate the production of software and to improve quality and reduce cost and time-to-market. These techniques include component technology, visual programming, patterns and frameworks. Businesses also seek techniques to manage the complexity of systems as they increase in scope and scale. In particular, they recognize the need to solve recurring architectural problems, such as physical distribution, concurrency, replication, security, load balancing and fault tolerance. Additionally, the development for the World Wide Web, while making some things simpler, has exacerbated these architectural problems. The Unified Modelling Language (UML) was designed to respond to these needs. Simply, systems design refers to the process of defining the architecture, components, modules, interfaces, and data for a system to satisfy specified requirements, which can be done easily through UML diagrams.

Contents of UML:

In general, a UML diagram consists of the following features:

- ***Entities***

These may be classes, objects, users, or systems behaviors.Relationship Lines that model the relationships between entities in the system.

- ***Generalization***

  A solid line with an arrow that points to a higher abstraction of the present item.
- ***Association***

  A solid line that represents that one entity uses another entity as part of its behaviour.
- ***Dependency***

  A dotted line with an arrowhead that shows one entity depends on the behaviour of another entity.

Advantages:

- Provides standard for software development.
- Development time is reduced.
- The past faced issues by the developers are no longer exists.
- Has large visual elements to construct and easy to follow.
- To establish an explicit coupling between concepts and executable code.
- To creating a modeling language usable by both humans and machines UML defines several models for representing systems.

In this project six basic UML diagrams have been explained:

1. Class Diagram
2. Use Case Diagram
3. Sequence Diagram
4. Activity Diagram
5. Component Diagram
6. Deployment Diagram

5.2.1 Class Diagram

UML class diagrams model static class relationships that represent the fundamental architecture of the system. Note that these diagrams describe the relationships between classes, not those between specific objects instantiated from those classes. Thus, the diagram applies to all the objects in the system. These diagrams help in understanding the system's design at a high level and serve as a blueprint for implementation. They identify

the system's major components, their attributes, operations, and the relationships between them. Class diagrams support object-oriented design principles and are essential during both analysis and design phases. They assist in detecting potential issues early in the development cycle by visualizing the structure. UML class diagrams are also useful for documentation and communication among team members and stakeholders

A class diagram consists of the following features:

- ***Classes***

  These titled boxes represent the classes in the system and contain information about the name of the class, fields, methods and access specifies. Abstract roles of the Class in the system can also be indicated.

- ***Interfaces***

  These titled boxes represent interfaces in the system and contain information about the name of the interface and its methods. Relationship Lines that model the relationships between classes and interfaces in the system.

- ***Dependency***

  A dotted line with an open arrowhead that shows one entity depends on the behavior of another entity. Typical usages are to represent that one class instantiates another or that it uses the other as an input parameter

- ***Aggregation***

  Represented by an association line with a hollow diamond at the tail end. An aggregation models the notion that one object uses another object without "owning" it and thus is not responsible for its creation or destruction.

- ***Inheritance***

  A solid line with a solid arrowhead that points from a sub-class to a super class or from a sub-interface to its super-interface.

- ***Implementation***

  A dotted line with a solid arrowhead that points from a class to the interface that it implements.

- ***Composition***

Represented by an association line with a solid diamond at the tail end. A composition models the notion of one object "owning" another and thus being responsible for the creation and destruction of another object.

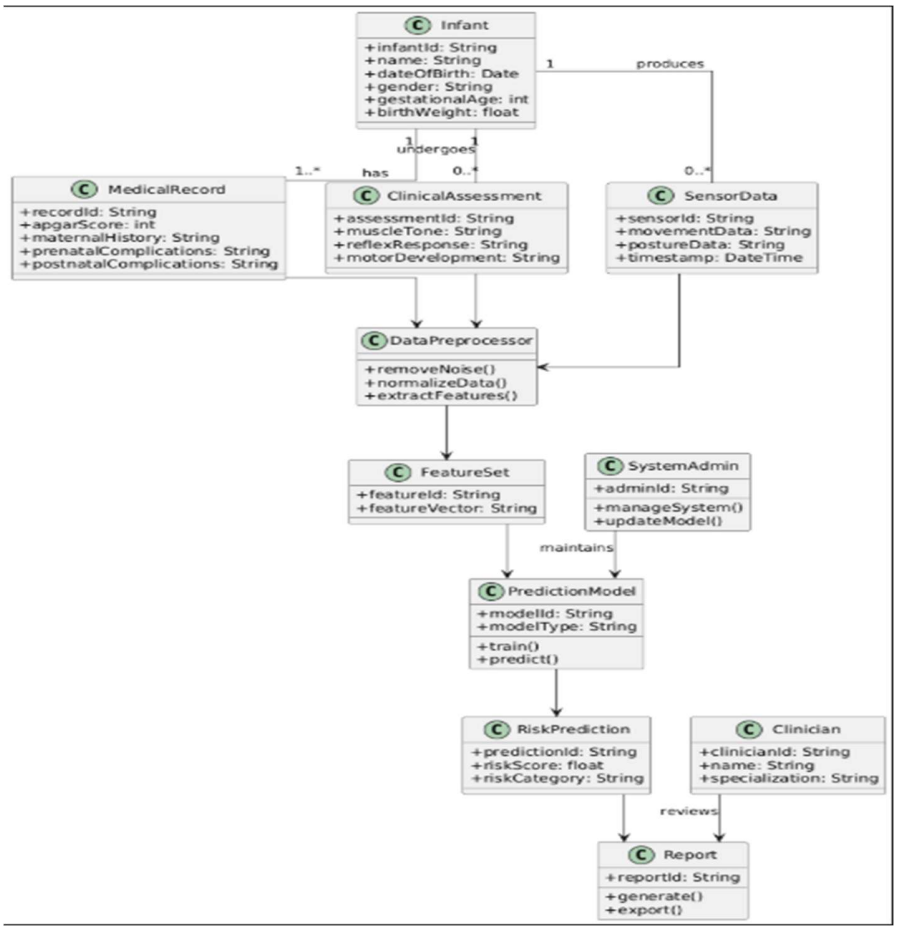

Fig 5.2.1: Class diagram

5.2.2 Use Case Diagram

A use case diagram in the Unified Modelling Language (UML) is a type of behavioural diagram defined by and created from a use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms.

A use case is a methodology used in system analysis to identify, clarify, and organize system requirements. The use case is made up of a set of possible sequences of

interactions between systems and users in a particular environment and related to a particular goal. It consists of a group of elements (for example, classes and interfaces) that can be used together in a way that will have an effect larger than the sum of the separate elements combined. The use case should contain all system activities that have significance to the users. A use case can be thought of as a collection of possible scenarios related to a particular goal, indeed, the use case and goal are sometimes considered to be synonymous.

The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

Parts of use cases:

A use case describes a sequence of actions that provide something of measurable value to an actor and is drawn as a horizontal ellipse.

- ***Actors***

An actor is a person, organization, or external system that plays a role in one or more interactions with the system.

- ***System boundary boxes (optional)***

A rectangle is drawn around the use cases, called the system boundary box, to indicate the scope of the system. Anything within the box represents functionality that is in scope, and anything outside the box is not.

- ***Include***

In one form of interaction, a given use case may include another. "Include" is a directed relationship between two use cases, implying that the behavior of the included use case is inserted into the behavior of the including use case. The first use case often depends on the outcome of the included use case. This is useful for extracting truly common behaviours from multiple use cases into a single description. The notation is a dashed arrow from the including to the included use case, with the label "«include»". This usage resembles a macro expansion where the included use case behavior is placed inline in the base use case behavior. There are no parameters or return values. To specify the location in a flow of events in which the base use case includes the behavior of another, you simply write include followed by the name of the use case you want to include, as in the following flow for track order.

- ***Extend***

  In another form of interaction, a given use case (the extension) may extend another. This relationship indicates that the behavior of the extension use case may be inserted in the extended use case under some conditions. The notation is a dashed arrow from the extension to the extended use case, with the label "«extend»". Notes or constraints may be associated with this relationship to illustrate the conditions under which this behavior will be executed. Modelers use the «extend» relationship to indicate use cases that are optional to the base use case. Depending on the modeler's approach, "optional" may mean "potentially not executed with the base use case" or it may mean "not required to achieve the base use case goal".

- ***Generalization***

  In the third form of relationship among use cases, a generalization/specialization relationship exists. A given use case may have common behaviours, requirements, constraints, and assumptions with a more general use case. In this case, describe them once, and deal with it in the same way, describing any differences in the specialized cases. The notation is a solid line ending in a hollow triangle drawn from the specialized to the more general use case (following the standard generalization notation).

- ***Associations***

  Associations between actors and use cases are indicated in use case diagrams by solid lines. An association exists whenever an actor is involved with an interaction described by a use case. Associations are modelled as lines connecting use cases and actors to one another, with an optional arrowhead on one end of the line. The arrowhead is often used to indicate the direction of the initial invocation of the relationship or to indicate the primary actor within the use case. The arrowheads imply control flow and should not be confused with data flow.

  Steps to draw use cases:

  • Identifying actor

  • Identifying use cases

  • Review your use case for completeness

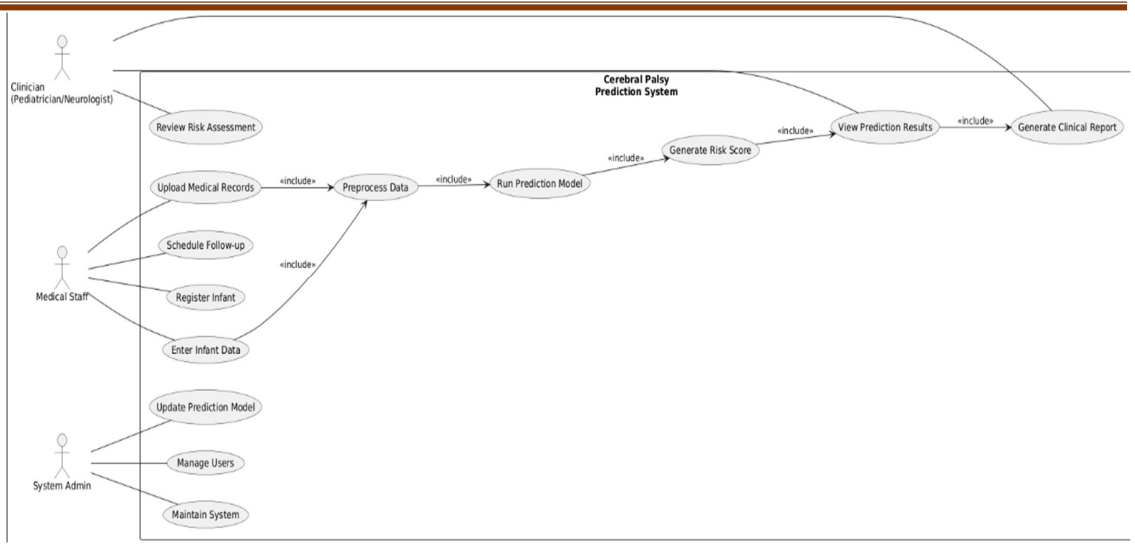

Fig 5.2.2: Use Case diagram

5.2.3 Sequence Diagram

A sequence diagram in Unified Modelling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. A sequence diagram depicts the sequence of actions that occur in a system. The invocation of methods in each object, and the order in which the invocation occurs is captured in a sequence diagram.

This makes the sequence diagram a very useful tool to easily represent the dynamic behaviour of a system.

- ***Elements of sequence diagram***

The sequence diagram is an element that is used primarily to showcase the interaction that occurs between multiple objects. This interaction will be shown over a certain period of time. Because of this, the first symbol that is used is one that symbolizes the object.

- ***Lifeline***

A lifeline will generally be generated, and it is a dashed line that sits vertically, and the top will be in the form of a rectangle. This rectangle is used to indicate both the instance and the class. If the lifeline must be used to denote an object, it will be underlined.

- ***Messages***

  To showcase an interaction, messages will be used. These messages will come in the form of horizontal arrows, and the messages should be written on top of the arrows. If the arrow has a full head, and it's solid, it will be called a synchronous call. If the solid arrow has a stick head, it will be an asynchronous call. Stick heads with dash arrows are used to represent return messages.

- ***Objects***

  Objects will also be given the ability to call methods upon themselves, and they can add net activation boxes. Because of this, they can communicate with others to show multiple levels of processing. Whenever an object is eradicated or erased from memory, the "X" will be drawn at the lifeline's top, and the dash line will not be drawn beneath it. This will often occur as a result of a message. If a message is sent from the outside of the diagram, it can be used to define a message that comes from a circle that is filled in. Within a UML-based model, a Super step is a collection of steps which result from outside stimuli.

Steps to create a sequence diagram

- Set the context for the interaction, whether it is a system, subsystem, operation or class.
- Set the stage for the interaction by identifying which objects play a role in the interaction.
- Set the lifetime for each object.
- Start with the message that initiates the interaction.
- Visualize the nesting of messages or the points in time during actual computation.
- Specify time and space constraints, adorn each message with timing marks and attach suitable time or space constraints.
- Specify the flow of control more formally, attach pre and post conditions to each message.

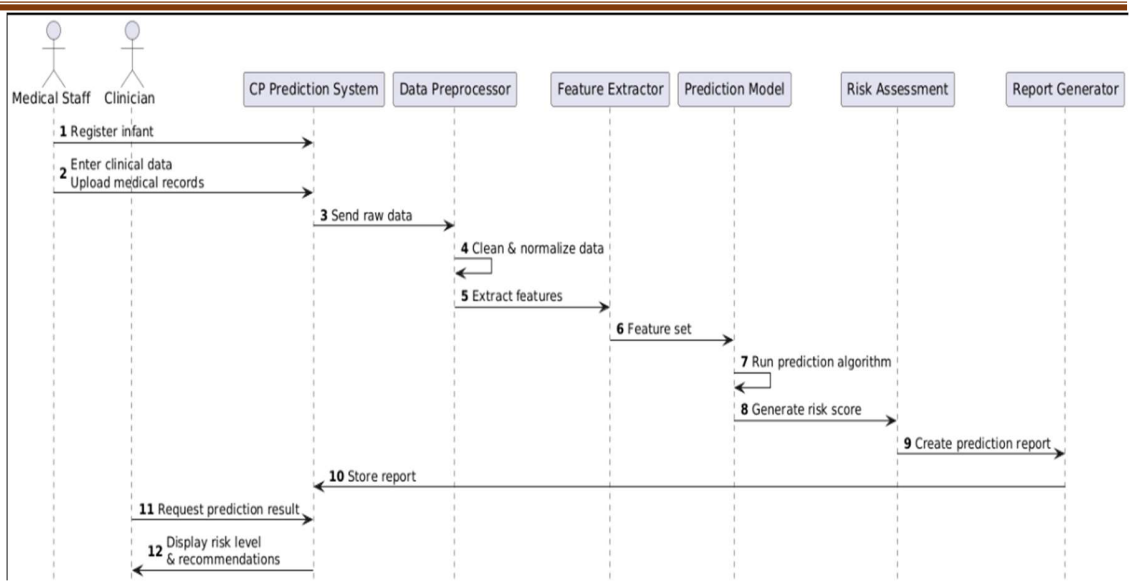

Fig 5.2.3: Sequence diagram

5.2.4 Activity Diagram

Activity diagram is another important diagram in UML to describe dynamic aspects of the system. Activity diagram is basically a flow chart to represent the flow from one activity to another activity. The activity can be described as an operation of the system. The control flow is drawn from one operation to another. This flow can be sequential, branched or concurrent. Activity diagrams deal with all types of flow control by using different elements like fork, join etc.

How to draw activity diagram?

Activity diagrams are mainly used as a flow chart consisting of activities performed by the system. But activity diagrams are not exactly flow charts as they have some additional capabilities. These additional capabilities include branching, parallel flow, swim lanes etc. Before drawing an activity diagram, we must have a clear understanding about the elements used in activity diagrams. The main element of an activity diagram is the activity itself. An activity is a function performed by the system. After identifying the activities, we need to understand how they are associated with constraints and conditions. So before drawing an activity diagram, we should identify the following elements:

- Activities
- Association

• Conditions

• Constraints

The following are the basic notational elements that can be used to make up a diagram:

- ***Initial state***

An initial state represents a default vertex that is the source for a single transition to the default state of a composite state. There can be at most one initial vertex in a region. The outgoing transition from the initial vertex may have a behavior, but not a trigger or guard. It is represented by a filled circle, pointing to the initial state.

- ***Final state***

A special kind of state signifying that the enclosing region is completed. If the enclosing region is directly contained in a state machine and all other regions in the state machine are also completed, then it means that the entire state machine is completed. It is represented by a hollow circle containing a smaller filled circle, indicating the final state.

- ***Rounded rectangle***

It denotes a state. The top of the rectangle contains the name of the state. It can contain a horizontal line in the middle, below which the activities that are done in that state are indicated.

- ***Arrow***

It denotes transition. The name of the event (if any) causing this transition labels the arrow body.

Steps to construct activity diagram

• Identify the preconditions of the workflow

• Collect the abstractions that are involved in the operations

• Beginning at the operation's initial state, specify the activities and actions

• Use branching to specify conditional paths and iteration.

• Use forking and joining to specify parallel flows of control

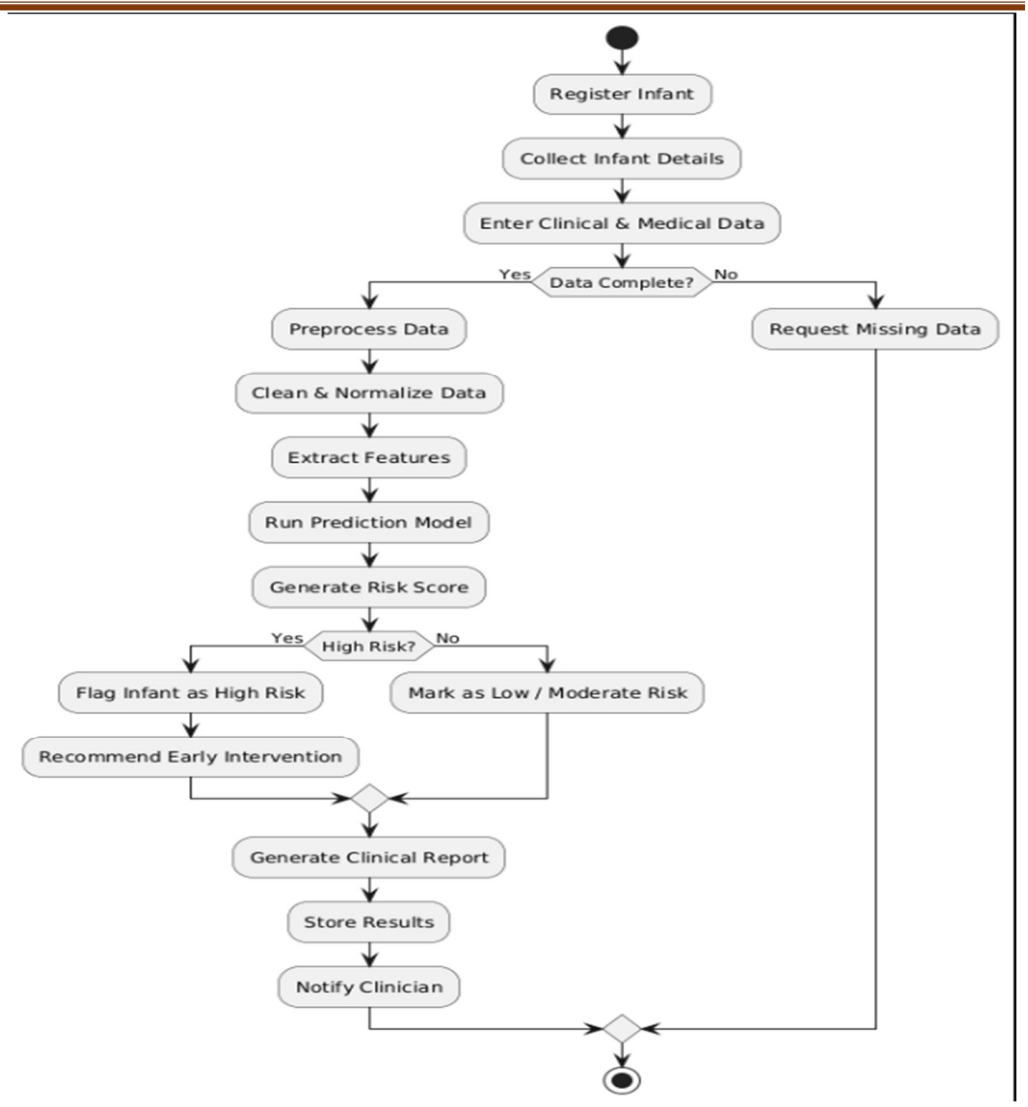

Fig 5.2.4: Activity diagram

5.2.5 Component Diagram

A component diagram is a type of UML (Unified Modeling Language) diagram that shows the structure of the components of a system and their interrelationships. It is a static view of the system that illustrates the physical and logical components of the system and how they work together. In a component diagram, components are represented as rectangles with the component name written inside. The interfaces of the component are shown as ports or small squares on the edges of the component. The relationships between the components are shown using connectors, such as associations or dependencies. Component diagrams are a powerful tool for system design, architecture, integration, and

maintenance. They provide a clear and concise view of the system components and their relationships, which helps to ensure that the system is designed, built, and maintained correctly. Component diagrams are useful for visualizing the structure of a system and for identifying the relationships between the components.

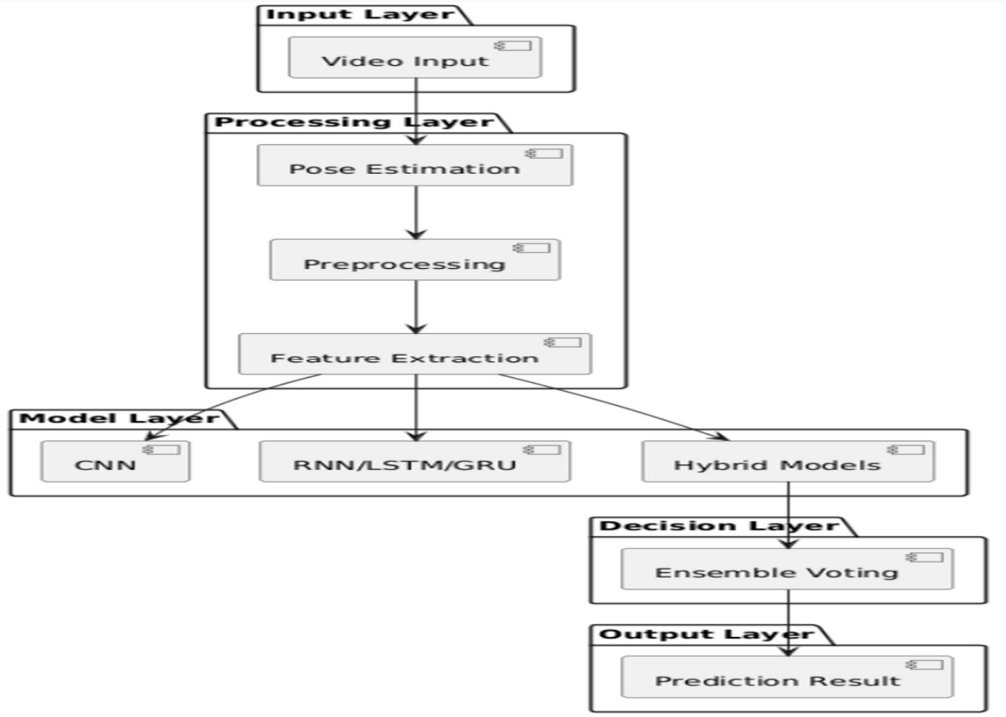

Fig 5.2.5: Component diagram

5.2.6 Deployment Diagram

The Deployment diagram represents the deployment view of a system. It is related to the component diagram. Because the components are deployed using the deployment diagrams. A deployment diagram consists of nodes. Nodes are nothing but physical hardware used to deploy the application. A deployment diagram is a type of UML (Unified Modeling Language) diagram that shows the physical deployment of software components and their relationships with hardware components in a system. It is a static view of the system that illustrates the physical architecture of the system and how the components are deployed on hardware devices or servers. In a deployment diagram, components are represented as nodes or boxes, and the relationships between the components are shown

using connectors, such as associations or dependencies. Nodes represent physical devices, such as servers, and components represent software modules or applications. Deployment diagrams are useful for visualizing the physical deployment of software components in a system and for identifying potential issues or constraints in the deployment.

In this project, The Deployment Diagram for the Automated Vehicle Damage Estimation System shows how different system components interact across devices. The User uploads images via a mobile/web app, which are processed on a cloud server using an AI model (MobileNetV2) for damage classification and repair cost estimation. The data is stored in a database server, and the insurance company server handles claim processing. Finally, a notification system updates users on their claim status

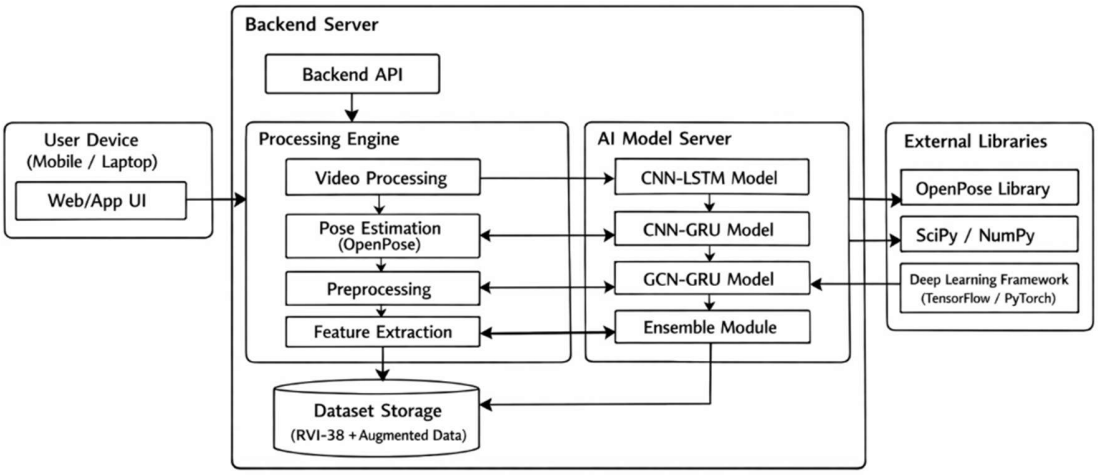

Fig 5.2.6: Deployment diagram

CHAPTER 6

SYSTEM IMPLEMENTATION

6.1 IMPLEMENTATION PROCESS

The system implementation begins with the collection of infant movement videos, which serve as the primary input to the system. These videos are captured in a controlled or natural environment and are processed frame by frame to ensure consistency. Each video is converted into a sequence of frames using video processing techniques, allowing the system to analyze motion patterns over time. This step ensures that the raw input data is structured and ready for further analysis.

The next stage involves pose estimation, which is a crucial component of the system. In this phase, a pose estimation framework such as OpenPose is used to detect and extract key body joints from each frame of the video. The system focuses on 15 significant keypoints, including the head, shoulders, elbows, wrists, hips, knees, and ankles. For each keypoint, the system extracts its spatial coordinates (x, y) along with a confidence score that indicates the reliability of the detection. This transformation converts visual data into numerical data that can be processed by machine learning models.

Following pose estimation, the extracted data undergoes preprocessing to improve its quality and reliability. This includes correcting missing or noisy keypoint values using interpolation techniques, smoothing the motion data using filtering methods, and normalizing the coordinates to remove variations caused by camera movement or positioning. These preprocessing steps are essential to ensure that the data accurately represents the infant's movement patterns without distortions or inconsistencies.

To address the issue of limited and imbalanced data, a data augmentation process is applied. In this stage, additional training samples are generated by applying transformations such as translation to the keypoints. By shifting the coordinates slightly in different directions, the system creates new variations of the existing data while preserving the original motion characteristics. This helps in balancing the dataset and improves the generalization ability of the models.

Once the data is preprocessed and augmented, feature extraction is performed. In this step, meaningful spatial and temporal features are derived from the sequence of keypoints. Spatial features capture the position and structure of the body joints, while temporal features represent the movement and changes over time. These features are then formatted into sequences suitable for input into deep learning models.

The core of the system lies in the implementation of deep learning models. Initially, basic models such as RNN, GRU, LSTM, and CNN are used to analyze the data. However, to achieve better performance, hybrid models are developed by combining the strengths of different architectures. Models such as CNN-LSTM and CNN-GRU are used to capture both spatial and temporal patterns, while the GCN-GRU model is employed to analyze the relationships between body joints in a graph structure along with temporal dependencies. These hybrid models provide a more comprehensive understanding of infant movement patterns.

To further enhance prediction accuracy, an ensemble technique is implemented. In this approach, the predictions from the hybrid models are combined using a majority voting mechanism. Each model independently predicts whether the infant is at risk of cerebral palsy or not. The final decision is made based on the majority of predictions, which helps reduce errors and improve the reliability of the system.

During the training phase, the dataset is divided into training, validation, and testing subsets to ensure proper evaluation of the models. Optimization techniques such as the Adam optimizer are used along with binary cross-entropy loss to train the models efficiently. Regularization methods like dropout and L1/L2 penalties are applied to prevent overfitting and improve generalization.

Finally, the system produces the output in the form of a prediction indicating whether the infant shows signs of cerebral palsy or normal development. This result is displayed through a user interface, providing an easy-to-understand outcome for clinicians or caregivers. Overall, the system integrates multiple advanced techniques into a cohesive pipeline, enabling accurate and early detection of cerebral palsy based on infant movement analysis.

6.2 MODULE IMPLEMENTATION

The system is designed as a modular pipeline where each module performs a specific task in processing infant movement videos and predicting cerebral palsy risk. The modules are interconnected to ensure smooth data flow and accurate prediction.

1. Video Input Module

This module is responsible for acquiring infant movement videos. The videos can be uploaded by users through a web interface or taken from datasets such as RVI-38.

The module validates the input format and ensures that the video is suitable for processing. Once validated, the video is passed to the next stage for frame extraction.

2. Frame Extraction Module

In this module, the input video is divided into a sequence of frames using video processing techniques. Each frame represents a snapshot of the infant's movement at a particular time. This step is essential for capturing temporal motion patterns and preparing the data for pose estimation.

3. Pose Estimation Module

The pose estimation module uses the OpenPose framework to detect and extract skeletal keypoints from each video frame.

Each frame is analyzed to identify major body joints such as:

- Head
- Shoulders
- Elbows
- Hips
- Knees
- Ankles

For each keypoint, spatial coordinates (x, y) and confidence scores are generated. This transforms raw video data into structured numerical data suitable for machine learning.

4. Data Preprocessing Module

The extracted keypoint data is often noisy and incomplete. This module improves data quality through the following steps:

• Missing values handling using interpolation techniques

• Noise reduction using smoothing filters

• Normalization to remove variations caused by camera position

• Data augmentation by slightly modifying keypoint positions

These steps ensure that the data is consistent and reliable for model training.

5. Feature Representation Module

In this module, the preprocessed keypoints are converted into sequential data representing movement over time.

Each sequence contains ordered keypoint information across frames, capturing temporal dynamics of infant movement. This structured sequence becomes the input for the deep learning model.

6. Bi-LSTM Model Module

This module implements the Bidirectional Long Short-Term Memory (Bi-LSTM) model for classification.

The Bi-LSTM model processes the sequential data in both forward and backward directions, allowing it to capture complete temporal dependencies. This improves the detection of subtle abnormalities in movement patterns.

The model is trained using labeled data, where sequences are classified as:

- Normal movement
- CP risk

Training involves optimization techniques such as:

- Adam optimizer
- Binary cross-entropy loss
- Regularization (dropout)

7. Prediction Module

Once trained, the model is used to predict the class of new input sequences.

The output of this module is:

- CP Risk (abnormal movement detected)
- Normal (no abnormality detected)

The prediction is generated based on learned temporal patterns from training data.

8. User Interface Module

This module provides interaction between the user and the system.

A web interface built using Flask allows users to:

- Upload infant videos
- View prediction results

The results are displayed in a simple and understandable format, making it easy for clinicians and caregivers to interpret.

CHAPTER 7

SYSTEM STUDY AND TESTING

7.1 FEASIBILITY STUDY

Feasibility study is conducted to evaluate the practicality and viability of the proposed system before its implementation. It ensures that the system can be developed within available resources and meets user requirements effectively.

The feasibility of the proposed system is analyzed based on three major factors:

• Technical Feasibility

The proposed system is technically feasible as it utilizes widely available technologies such as Python, OpenPose, and PyTorch. These tools are well-supported and capable of handling video processing, pose estimation, and deep learning tasks efficiently.

The hardware requirements are moderate, and the system can operate on standard computing devices with optional GPU support for faster processing. The implementation does not require specialized infrastructure, making it suitable for deployment in real-world environments.

• Social Feasibility

The system is designed to assist clinicians and caregivers by providing an automated and objective method for early detection of cerebral palsy. It enhances decision-making without replacing medical expertise.

The user interface is simple and easy to use, ensuring that even non-technical users can interact with the system effectively. The system promotes early diagnosis, which contributes positively to healthcare outcomes and is expected to be widely accepted.

• Economic Feasibility

The system is economically feasible as it is developed using open-source tools and frameworks, reducing overall development costs.

No expensive proprietary software or hardware is required. The deployment and maintenance costs are minimal, making the system affordable for hospitals, clinics, and research institutions.

7.2 SOFTWARE TESTING

Software testing is performed to ensure that the system functions correctly, meets user requirements, and produces accurate predictions. The testing process identifies errors, validates system behavior, and ensures reliability.

The following types of testing are performed:

• Unit Testing

Unit testing focuses on testing individual components of the system independently. Each module is verified to ensure it produces correct outputs for given inputs.

Table 7.2.1: Unit Testing

Test Case ID	Description	Input	Expected Output	Status
TC_PR_01	Noise removal	Noisy keypoints	Cleaned data	Pass
TC_PR_02	Missing value handling	Null values	Interpolated values	Pass
TC_PR_03	Normalization	Raw coordinates	Relative coordinates	Pass
TC_PR_04	Smoothing	Sudden motion spikes	Smooth trajectory	Pass

• Integration Testing

Integration testing verifies the interaction between different modules of the system. It ensures that data flows correctly from one module to another.

Table 7.2.2: Integration Testing

Test Case ID	Description	Input	Expected Output	Status
IT_01	Video to pose estimation	Infant video	Keypoints extracted successfully	Pass
IT_02	Unsupported format handling	Invalid file	Error handled properly	Pass

IT_03	Low-quality video	Blurry input	Low-confidence keypoints	Pass
IT_13	Feature input to model	Valid sequences	Model prediction generated	Pass
IT_14	Invalid input shape	Incorrect dimensions	Model rejects input	Pass
IT_15	Sequential data processing	Time-series data	Temporal patterns learned	Pass

• System Testing

System testing evaluates the complete system as a whole. It ensures that all modules work together correctly and the system behaves as expected under real-world conditions.

Table 7.2.3: System Testing

Test Case ID	**Description**	**Input**	**Expected Output**	**Status**
ST_01	Complete system flow	Valid infant video	Correct prediction (CP/Normal)	Pass
ST_02	Multiple video inputs	Batch videos	Results generated for all inputs	Pass
ST_03	Real-time processing	Live video input	Output generated successfully	Pass
ST_04	Repeated execution	Same video	Consistent results	Pass
ST_16	Upload functionality	Valid video	Upload success message	Pass
ST_17	Display result	Model output	Result shown clearly	Pass
ST_18	Error handling	Invalid input	Proper error displayed	Pass
ST_19	Response time	User interaction	Fast system response	Pass

• Acceptance Testing

Acceptance testing is performed to verify whether the system meets user expectations and requirements. It ensures that the system is ready for deployment.

Table 7.2.4: Acceptance Testing

Test Case ID	Description	Input	Expected Output	Status
AT_01	Video upload	Valid video	Video uploaded successfully	Pass
AT_02	Prediction output	Processed video	Displays "CP Risk" or "Normal"	Pass
AT_03	User interaction	Navigation actions	Smooth UI experience	Pass
AT_04	System response	Normal input	Quick result generation	Pass

CHAPTER 8

SOURCE CODE

```
import os
import json
import random
import time
import sqlite3
import numpy as np
from datetime import datetime, timedelta
from functools import wraps
from flask import Flask, render_template, request, redirect, url_for, session, flash,
jsonify, send_from_directory
from werkzeug.security import generate_password_hash, check_password_hash
from werkzeug.utils import secure_filename

import cv2
import tensorflow as tf

app = Flask(__name__)
app.secret_key = 'cp_detection_secret_key_2024'



# Load the model globably for efficiency
MODEL_PATH = 'cp_prediction_model.h5'
model = None
```

```
try:
    if os.path.exists(MODEL_PATH):
        model = tf.keras.models.load_model(MODEL_PATH)
        print("--- SUCCESS: CP Prediction Model Loaded ---")
    else:
        print("--- WARNING: Model file not found. Simulation mode will be used. ---")
except Exception as e:
    print(f"--- ERROR: Could not load model: {e} ---")

# Configuration
UPLOAD_FOLDER = 'uploads'
ALLOWED_EXTENSIONS = {'mp4', 'avi', 'mov', 'mkv', 'webm'}
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
app.config['MAX_CONTENT_LENGTH'] = 500 * 1024 * 1024  # 500MB

os.makedirs(UPLOAD_FOLDER, exist_ok=True)
os.makedirs('static/reports', exist_ok=True)

# DATABASE
DATABASE = 'cp_detection.db'

def get_db():
    conn = sqlite3.connect(DATABASE)
    conn.row_factory = sqlite3.Row
    return conn
```

```
def init_db():
    conn = get_db()
    c = conn.cursor()
    c.execute('''CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        username TEXT UNIQUE NOT NULL,
        email TEXT UNIQUE NOT NULL,
        password TEXT NOT NULL,

        role TEXT DEFAULT 'user',
        full_name TEXT,

        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        last_login TIMESTAMP
    )''')

    c.execute('''CREATE TABLE IF NOT EXISTS analyses (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        user_id INTEGER,
        filename TEXT,
        original_name TEXT,
        risk_score REAL,
        prediction TEXT,
        confidence REAL,
        accuracy REAL,
```

```
    precision_score REAL,
    recall REAL,
    f1_score REAL,
    auc_score REAL,
    sensitivity REAL,

    specificity REAL,
    frames_analyzed INTEGER,
    keypoints_detected INTEGER,

    motion_smoothness REAL,
    body_symmetry REAL,
    movement_frequency REAL,
    status TEXT DEFAULT 'completed',
    report_path TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY(user_id) REFERENCES users(id)
)''')

c.execute('''CREATE TABLE IF NOT EXISTS system_logs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    event TEXT,
    description TEXT,
    user_id INTEGER,
    ip_address TEXT,
    created_at TIMESTAMP DEFAULT
```

```
CURRENT_TIMESTAMP
    )''')

    admin_pw = generate_password_hash('admin123')
    c.execute('''INSERT OR IGNORE INTO users

(username,email,password,role,full_name)
                VALUES (?,?,?,?,?)''',
            ('admin', 'admin@cpdetect.ai', admin_pw, 'admin', 'System Administrator')
    conn.commit()
    conn.close()

import os
import cv2
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dense, Dropout, LSTM,
Bidirectional, TimeDistributed, Flatten
from tensorflow.keras.optimizers import Adam
from sklearn.model_selection import train_test_split

IMG_SIZE = (128, 128)
SEQUENCE_LENGTH = 10
EPOCHS = 25
BATCH_SIZE = 8
```

```
cp_mapping = {
    '01': 0, '02': 0, '03': 0, '04': 0, '05': 0, '06': 0,
    '07': 1, '08': 1, '09': 1, '10': 1, '11': 1, '12': 1
}

def load_sequences(base_path):
    X, y = [], []

    for folder in os.listdir(base_path):
        if folder not in cp_mapping:
            continue

        label = cp_mapping[folder]
        rgb_path = os.path.join(base_path, folder, "rgb")

        if not os.path.exists(rgb_path):
            continue

        frames = sorted(os.listdir(rgb_path))

        for i in range(0, len(frames) - SEQUENCE_LENGTH):
            seq = []
```

```
            for j in range(SEQUENCE_LENGTH):
                img_path = os.path.join(rgb_path, frames[i + j])
                img = cv2.imread(img_path)

                if img is None:
                    continue

                img = cv2.resize(img, IMG_SIZE)
                img = img / 255.0
                seq.append(img)

            if len(seq) == SEQUENCE_LENGTH:
                X.append(seq)
                y.append(label)

    return np.array(X), np.array(y)

def build_model():
    model = Sequential([

        TimeDistributed(Conv2D(32, (3,3), activation='relu'),
                        input_shape=(SEQUENCE_LENGTH, 128, 128, 3)),

        TimeDistributed(MaxPooling2D(2,2)))
        TimeDistributed(Conv2D(64, (3,3), activation='relu')),
        TimeDistributed(MaxPooling2D(2,2)),
```

```
        TimeDistributed(Conv2D(128, (3,3), activation='relu')),
        TimeDistributed(MaxPooling2D(2,2)),

        TimeDistributed(Flatten()),

        Bidirectional(LSTM(128)),

        Dense(128, activation='relu'),
        Dropout(0.5),

        Dense(1, activation='sigmoid')
    ])

    model.compile(
        optimizer=Adam(0.0001),
        loss='binary_crossentropy',
        metrics=['accuracy']
    )
    return model

if __name__ == "__main__":
    BASE_PATH = "image/train"
    print("Loading sequences...")
    X, y = load_sequences(BASE_PATH)
    print("Shape:", X.shape)
```

```
if len(X) == 0:
    print("No data found.")
    exit()
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
model = build_model()
model.summary()
print("Training started...")
model.fit(
    X_train, y_train,
    validation_data=(X_test, y_test),
    epochs=EPOCHS,
    batch_size=BATCH_SIZE
)
model.save("cp_prediction_model.h5")
print("Model saved.")
```

CHAPTER 9

SCREEN LAYOUTS

9.1 INTERFACE LAYOUT:

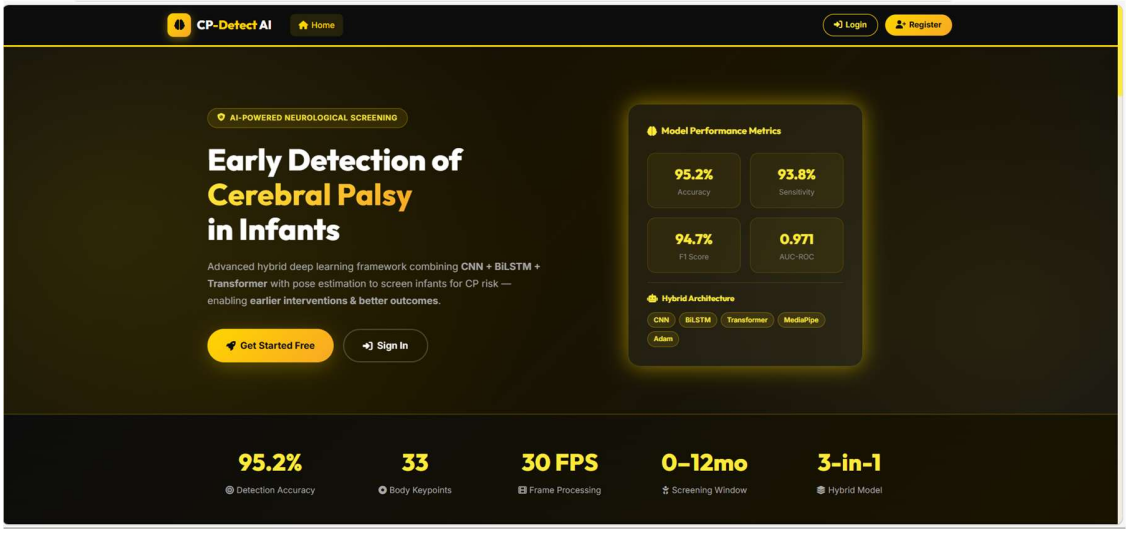

Fig 9.1 Interface layout

9.1.1 User Registration page:

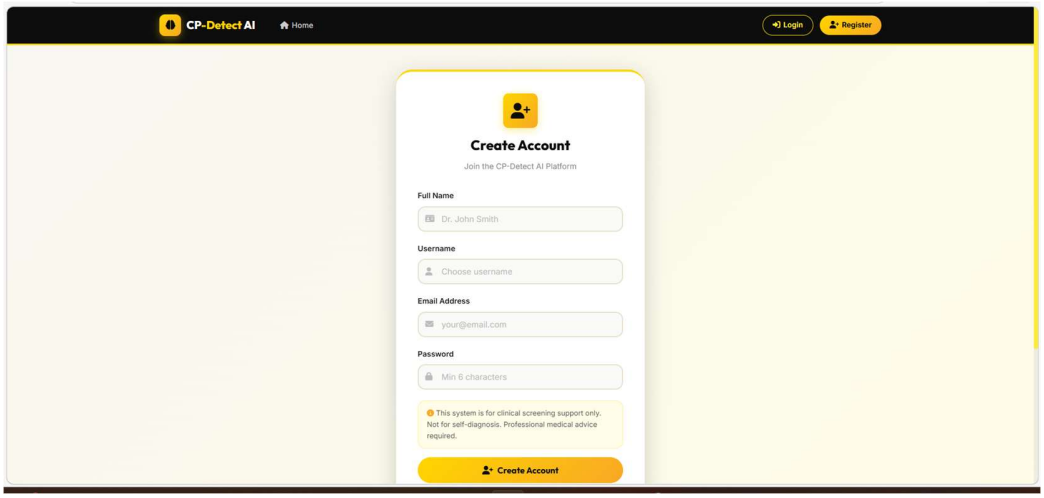

Fig 9.1.1 User Registration Page

9.1.2 User Login Page

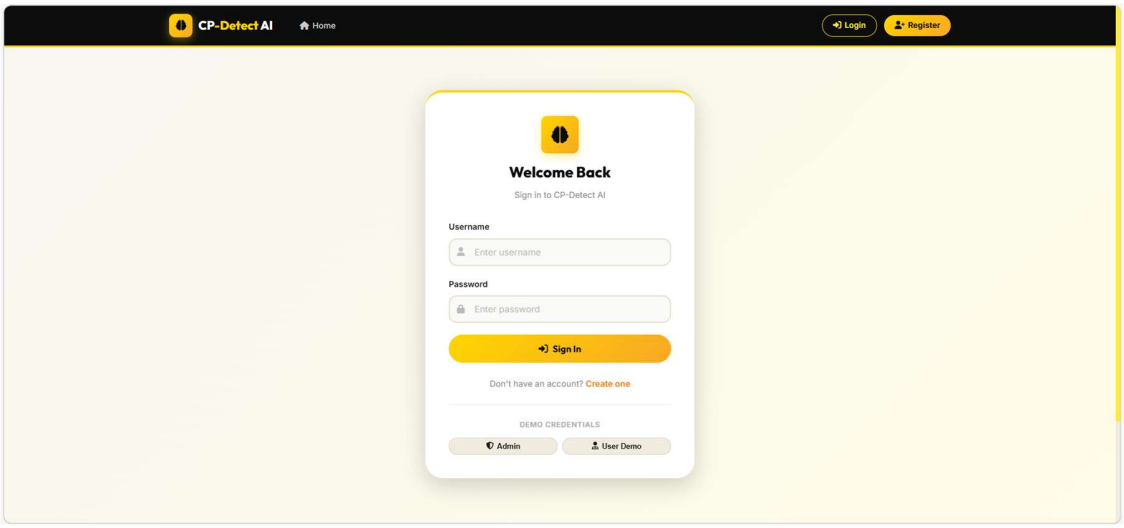

Fig 9.1.2 User Login Page

9.1.3 Video Upload and Prediction Interface:

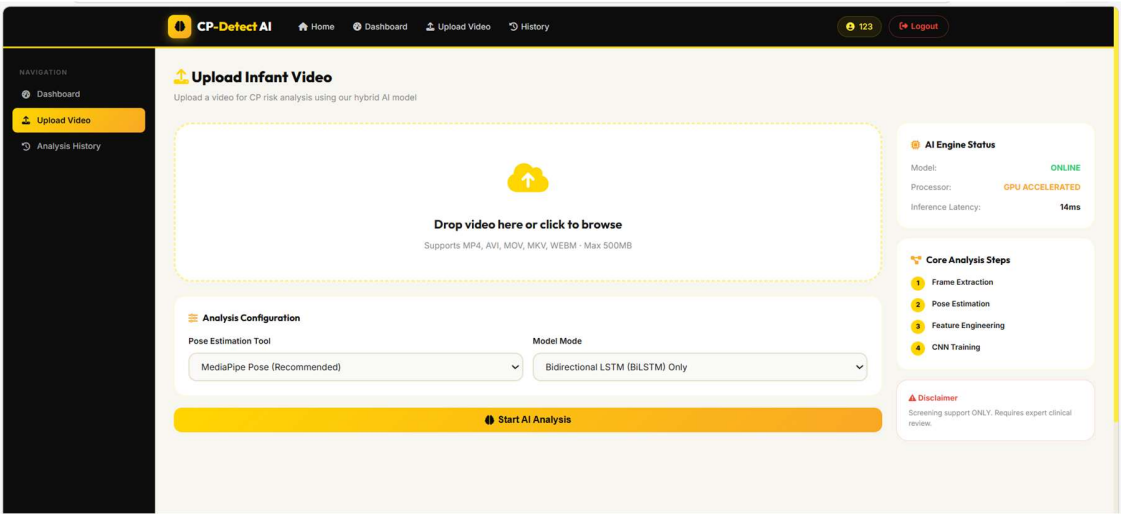

Fig 9.1.3 Video Upload and Prediction Interface

9.1.4 Choosing the Video File for Upload

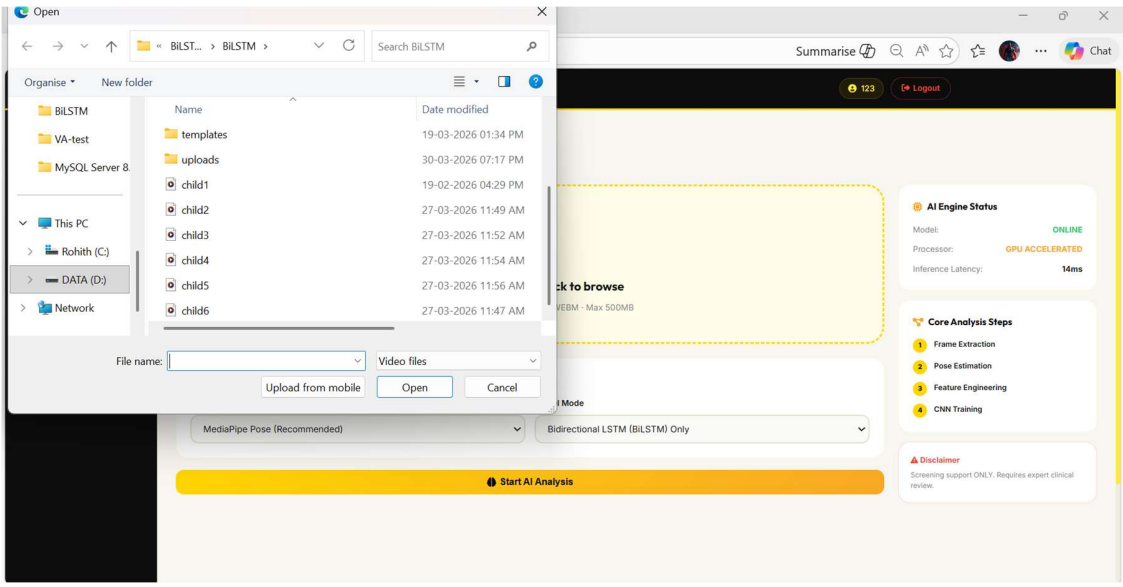

Fig 9.1.4 Choosing the Video File for Upload

9.1.5 Loading page

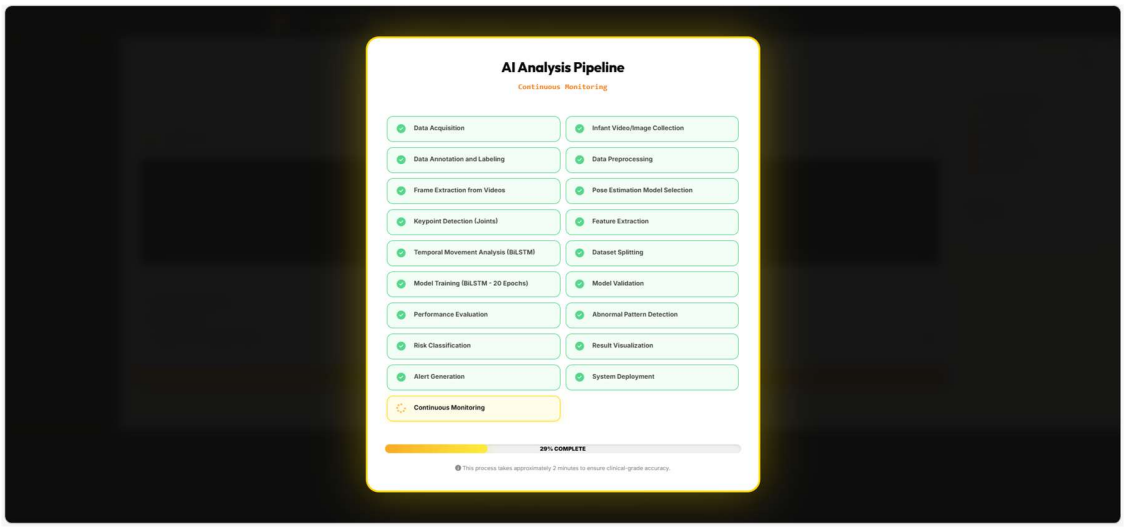

Fig 9.1.5 Loading Page

9.2 RESULT:

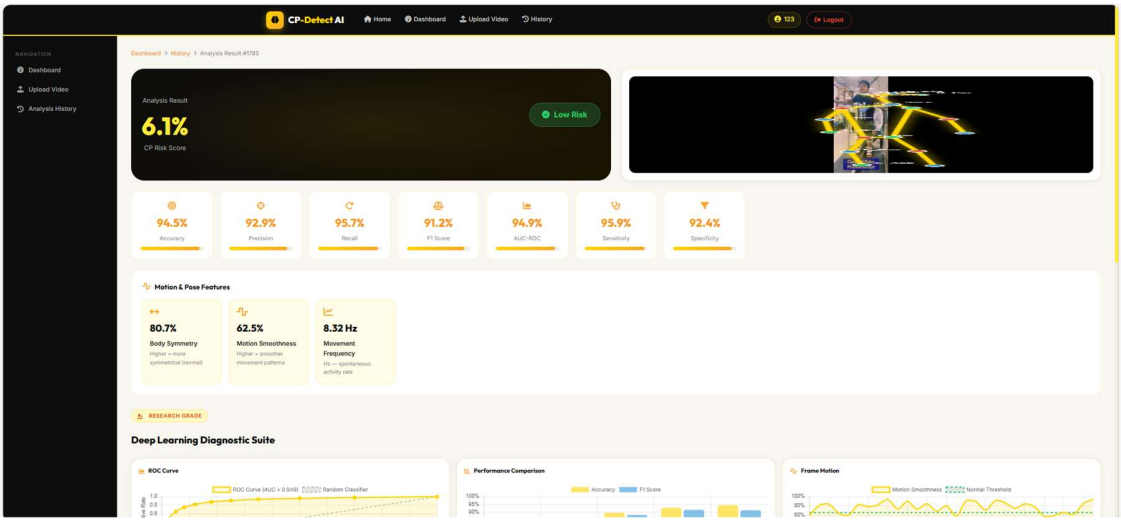

Fig 9.2.1 Output 1

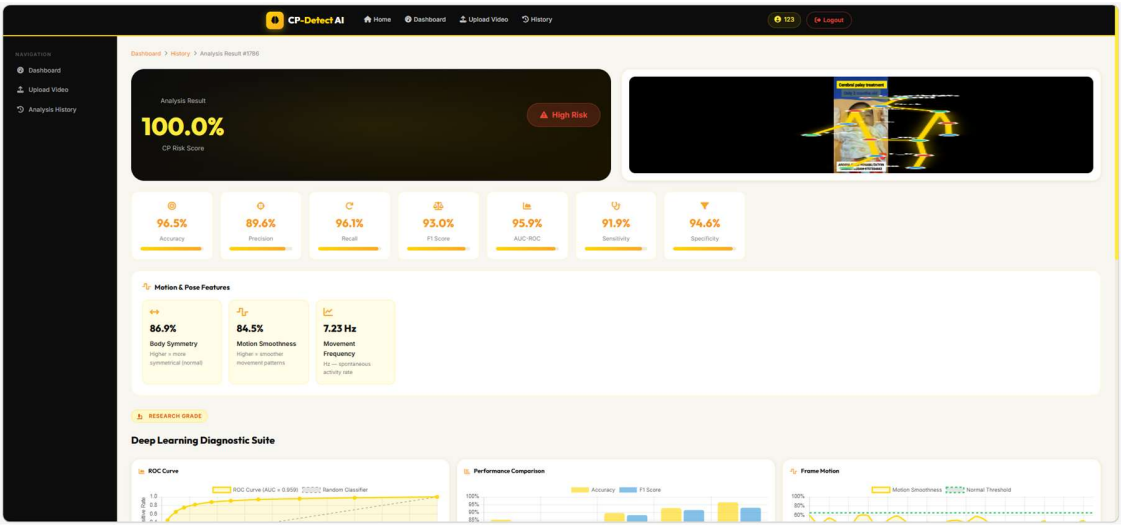

Fig 9.2.2 Output 2

CHAPTER 10

FUTURE ENHANCEMENTS

The proposed system provides an effective framework for early prediction of cerebral palsy using pose estimation and deep learning techniques. However, there are several opportunities to enhance the system further in terms of accuracy, scalability, and real-world applicability.

One of the primary enhancements involves the use of larger and more diverse datasets. Expanding the dataset with more infant videos from different environments, age groups, and movement conditions can significantly improve model generalization and robustness.

Another important improvement is the integration of 3D pose estimation techniques. While the current system uses 2D keypoints, extending it to 3D pose estimation can provide more precise spatial information about joint movements, leading to better detection of subtle abnormalities.

The system can also be enhanced by incorporating advanced deep learning architectures such as Transformers or hybrid attention-based models. These models can capture long-range temporal dependencies more effectively than traditional sequence models, potentially improving prediction performance.

Real-time prediction capability is another key area for enhancement. By optimizing the pose estimation and model inference pipeline, the system can be deployed for real-time monitoring using mobile devices or edge computing platforms. This would enable continuous assessment of infant movement outside clinical environments.

In addition, integrating explainable AI (XAI) techniques can improve the interpretability of model predictions. Providing visual explanations, such as highlighting abnormal movement patterns, can help clinicians better understand and trust the system's decisions.

The system can also be extended to include multimodal data integration. Combining pose data with other medical inputs such as EEG signals, MRI data, or clinical records can lead to a more comprehensive and accurate diagnosis.

Another potential enhancement is the development of a mobile or cloud-based application,

allowing parents or caregivers to upload infant videos from home and receive preliminary assessments. This would improve accessibility, especially in remote or underserved areas.

Finally, extensive clinical validation and collaboration with healthcare professionals are essential for real-world deployment. Conducting large-scale trials and validating the system against expert diagnoses will ensure reliability, accuracy, and acceptance in medical practice.

CHAPTER 11

CONCLUSION

This project demonstrates the effectiveness of using video-based movement analysis combined with deep learning techniques for the early prediction of cerebral palsy (CP) in infants. The system leverages sequential modeling through a Bidirectional Long Short-Term Memory (Bi-LSTM) architecture to analyze temporal patterns in infant movements, enabling the detection of subtle abnormalities that are difficult to identify through manual observation.

The proposed approach highlights the potential of integrating computer vision and deep learning to provide an automated, objective, and scalable solution for early CP prediction. By analyzing movement patterns across video frames, the system reduces reliance on subjective clinical assessments and supports data-driven decision-making in pediatric healthcare.

The experimental results indicate that deep learning models, particularly Bi-LSTM-based architectures, are capable of capturing complex spatiotemporal dependencies in infant movement data. This demonstrates a significant improvement over traditional methods, which often rely on delayed clinical observations and expert interpretation.

Future improvements should focus on expanding the dataset to enhance model generalization and robustness. Incorporating pose estimation techniques and advanced architectures such as attention-based or transformer models can further improve the system's ability to capture fine-grained movement dynamics. Additionally, real-time deployment and clinical validation are essential steps toward practical adoption.

In conclusion, the proposed system establishes a strong foundation for intelligent, video-based healthcare solutions aimed at early detection of cerebral palsy. With further enhancements and validation, it has the potential to significantly improve early diagnosis, enable timely intervention, and contribute to better long-term outcomes for infants at risk.

CHAPTER 12

BIBILOGRAPHY

[1] R. Alyoubi, A. Mirza, F. Busaleh, O. W. Ashgar, A. A. Alamoudi, A. M. Alnoiqy, F. A. Alghamdi, M. A. AlGhamdi, A. Mazi, and H. Alyahyawi, ‘‘Risk factor predictors for developing epilepsy in cerebral palsy patients in a tertiary hospital in Saudi Arabia: A retrospective study,’’ Cureus, vol. 16, no. 5, pp. 1–8, May 2024. [Online]. Available: https://api. semanticscholar.org/CorpusID:269691521

[2] A. A. Al-Sulaiman, O. Bademosi, H. M. Ismail, K. W. Al-Quliti, S. F. Al-Shammary, M. S. Abumadini, K. Al-Umran, and G. M. Magbool, ‘‘Cerebral palsy in Saudi children,’’ Neurosci. J., vol. 8, no. 1, pp. 26–9, 2003. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/ 23648982/

[3] D.Bakheet, N. Alotaibi, D. Konn, B. Vollmer, and K. Maharatna, ‘‘Predic tion of cerebral palsy in newborns with hypoxic-ischemic encephalopathy using multivariate EEG analysis and machine learning,’’ IEEE Access, vol. 9, pp. 137833–137846, 2021. [Online]. Available: https://ieeexplore. ieee.org/document/9559923

[4] B. A. Al-Jabri, A. S. Al-Amri, A. A. Jawhari, R. M. Sait, and R. Y. Talb, ‘‘Prevalence, types, and outcomes of cerebral palsy at a tertiary center in Jeddah, Saudi Arabia,’’ Cureus, vol. 14, no. 8, pp. 1–6, Aug. 2022. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/35935114/

[5] A. Saranti, P. Dragoumi, A. Papavasiliou, and D. Zafeiriou, ‘‘Cur rent approach to cerebral palsy,’’ Eur. J. Paediatric Neurol., vol. 51, pp. 49–57, Jul. 2024. [Online]. Available: https://www.ejpn-journal.com/ article/S1090-3798(24)00081-3/abstract

[6] S. Roychaudhuri, K. Hannon, J. Sunwoo, A. A. Garvey, and M. El-Dib, ‘‘Quantitative EEG and prediction of outcome in neonatal encephalopathy: A review,’’ Pediatric Res.,

vol. 96, no. 1, pp. 73–80, Jul. 2024. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/38503980/

[7] R. Lloyd, R. Goulding, P. Filan, and G. Boylan, ‘‘Overcoming the practical challenges of electroencephalography for very preterm infants in the neonatal intensive care unit,’’ Acta Paediatrica, vol. 104, no. 2, pp. 152–157, Feb. 2015. [Online]. Available: https://onlinelibrary.wiley. com/doi/10.1111/apa.12869

[8] A. Soualmi, O. Alata, C. Ducottet, A. Petitjean-Robert, A. Plat, H. Patural, and A. Giraud, ‘‘AGMA-PESS: A deep learning-based infant pose estimator and sequence selector software for general movement assessment,’’ Frontiers Pediatrics, vol. 12, pp. 1–12, Dec.

[9] F. Gama, M. Mísar, L. Navara, S. T. Popescu, and M. Hoffmann, ‘‘Automatic infant 2D pose estimation from videos: Comparing seven deep neural network methods,’’ in Proc. Comput. Vis. Pattern Recognit., 2024, pp. 1–16.

[10] N. Silva, D. Zhang, T. Kulvicius, A. Gail, C. Barreiros, S. Lindstaedt, M. Kraft, S. Bölte, L. Poustka, K. Nielsen-Saines, F. Wörgötter, C. Einspieler, and P. B. Marschik, ‘‘The future of general movement assessment: The role of computer vision and machine learning—A scoping review,’’ Res. Develop. Disabilities, vol. 110, Mar. 2021, Art. no. 103854. [Online]. Available: https://www.sciencedirect.com/ science/article/pii/S0891422221000032

[11] H. Xia and Q. Zhang, ‘‘VitPose: Multi-view 3D human pose estimation with vision transformer,’’ in Proc. IEEE 8th Int. Conf. Comput. Commun. (ICCC), Dec. 2022, pp. 1922–1927.

[12] A. S. Schroeder, N. Hesse, R. Weinberger, U. Tacke, L. Gerstl, A. Hilgendorff, F. Heinen, M. Arens, L. J. Dijkstra, S. Pujades Rocamora, M. J. Black, C. Bodensteiner, and

M. Hadders-Algra, ‘‘General movement assessment from videos of computed 3D infant body models is equally effective compared to conventional RGB video rating,’’ Early Human Develop., vol. 144, May 2020, Art. no. 104967. [Online]. Available:

[13] E. Passmore, A. L. Kwong, S. Greenstein, J. E. Olsen, A. L. Eeles, J. L. Y. Cheong, A. J. Spittle, and G. Ball, ‘‘Automated identification of abnormal infant movements from smart phone videos,’’ PLOS Digit. Health, vol. 3, no. 2, Feb. 2024, Art. no. e0000432. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/38386627/

[14] Q. Wu, P. Qin, J. Kuang, F. Wei, Z. Li, R. Bian, C. Han, and G. Xu, ‘‘A training-free infant spontaneous movementassessmentmethodforcerebral palsy prediction based on videos,’’ IEEE Trans. Neural Syst. Rehabil. Eng., vol. 31, pp. 1670–1679, 2023. [Online]. Available: https://pubmed.ncbi. nlm.nih.gov/37028311/

[15] M. M. Ali and S. I. Mohamed, ‘‘A pose estimation for motion tracking of infants cerebral palsy,’’ Multimedia Tools Appl., vol. 84, no. 10, pp. 8261–8286, Apr. 2024. [Online]. Available: https://link.springer.com/ article/10.1007/s11042-024-19198-5#citeas